

Class Vehiculo

[java.lang.Object](#)

[com.mycompany.pruebavehiculo.Vehiculo](#)

```
public class Vehiculo
```

```
extends Object
```

Representa un vehículo con su propietario, su kilometraje y un depósito de combustible.

Permite viajar (gastando combustible), repostar litros concretos, llenar el depósito y llevar la cuenta del dinero gastado. El precio del combustible es común a todos los vehículos ([actualizarPrecio\(double\)](#)).

Ejemplo de uso:

```
Vehiculo coche = new Vehiculo("Seat", "Ibiza", "1234ABC", "Ana", "12345678Z",  
coche.llenar();  
System.out.println(coche.viajar(100));
```

See Also:

[DNI](#)

Constructor Summary

Constructors

Constructor	Description
Vehiculo (String marca, String modelo, String matricula, String nombre, String dni, double kilometrosRecorridos, double capacidadDeposito)	Crea un vehículo con el depósito vacío.

Method Summary

All Methods

Static Methods

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
static String	actualizarPrecio (double nuevoPrecio)	Actualiza el precio del litro de combustible.
double	getCapacidadDeposito ()	Devuelve la capacidad del depósito.
DNI	getDni ()	Devuelve el DNI del propietario.

String	getMARCA()	Devuelve la marca del vehículo.
String	getMATRICULA()	Devuelve la matrícula del vehículo.
String	getMODELO()	Devuelve el modelo del vehículo.
String	getNombre()	Devuelve el nombre del propietario.
double	getNumKilometros()	Devuelve los kilómetros totales recorridos por el vehículo.
String	llenar()	Llena el depósito hasta su capacidad máxima y suma el coste de los litros añadidos al dinero gastado.
String	mostrarDatosIdentificativos()	Muestra los datos que identifican al vehículo.
String	mostrarEstadoVehiculo()	Muestra el estado actual del vehículo.
String	mostrarPropietario()	Muestra el propietario del vehículo.
String	repostar(double litros)	Reposta una cantidad de litros y suma su coste al dinero gastado.
void	setCapacidadDeposito (double capacidadDeposito)	Cambia la capacidad del depósito.
void	setDni(DNI dni)	Cambia el DNI del propietario.
void	setNombre(String nombre)	Cambia el nombre del propietario.
void	setNumKilometros (double numKilometros)	Cambia los kilómetros totales del vehículo.
String	viajar(double kmViajar)	Realiza un viaje de los kilómetros indicados.

Methods inherited from class **Object**

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

Vehiculo

```
public Vehiculo(String↗ marca,  
                String↗ modelo,  
                String↗ matricula,  
                String↗ nombre,  
                String↗ dni,  
                double kilometrosRecorridos,  
                double capacidadDeposito)
```

Crea un vehículo con el depósito vacío.

Parameters:

marca - marca del vehículo

modelo - modelo del vehículo

matricula - matrícula del vehículo

nombre - nombre del propietario

dni - DNI del propietario, con letra (por ejemplo "12345678Z")

kilometrosRecorridos - kilómetros que ya tiene el vehículo (no puede ser negativo)

capacidadDeposito - capacidad del depósito en litros (debe ser mayor que 0)

Throws:

[IllegalArgumentException[↗]](#) - si el DNI no es válido, los kilómetros son negativos o la capacidad no es positiva

Method Details

getMARCA

```
public String↗ getMARCA()
```

Devuelve la marca del vehículo.

Returns:

la marca

getMODELO

```
public String↗ getMODELO()
```

Devuelve el modelo del vehículo.

Returns:

el modelo

getMATRICULA

```
public String getMATRICULA()
```

Devuelve la matrícula del vehículo.

Returns:

la matrícula

getNombre

```
public String getNombre()
```

Devuelve el nombre del propietario.

Returns:

el nombre del propietario

getDni

```
public DNI getDni()
```

Devuelve el DNI del propietario.

Returns:

el DNI del propietario

getNumKilometros

```
public double getNumKilometros()
```

Devuelve los kilómetros totales recorridos por el vehículo.

Returns:

los kilómetros totales

getCapacidadDeposito

```
public double getCapacidadDeposito()
```

Devuelve la capacidad del depósito.

Returns:

la capacidad, en litros

setNombre

```
public void setNombre(String nombre)
```

Cambia el nombre del propietario.

Parameters:

nombre - el nuevo nombre del propietario

setDni

```
public void setDni(DNI dni)
```

Cambia el DNI del propietario.

Parameters:

dni - el nuevo DNI (no puede ser null)

Throws:

[IllegalArgumentException](#) - si el DNI es null

setNumKilometros

```
public void setNumKilometros(double numKilometros)
```

Cambia los kilómetros totales del vehículo.

Parameters:

numKilometros - los nuevos kilómetros totales (no puede ser negativo)

Throws:

[IllegalArgumentException](#) - si el valor es negativo

setCapacidadDeposito

```
public void setCapacidadDeposito(double capacidadDeposito)
```

Cambia la capacidad del depósito.

Parameters:

capacidadDeposito - la nueva capacidad en litros (debe ser mayor que 0 y no menor que el combustible que hay ahora en el depósito)

Throws:

[IllegalArgumentException](#) - si la capacidad no es positiva o es menor que el combustible actual

mostrarDatosIdentificativos

```
public String mostrarDatosIdentificativos()
```

Muestra los datos que identifican al vehículo.

Returns:

una cadena con la marca, el modelo y la matrícula

mostrarEstadoVehiculo

```
public String mostrarEstadoVehiculo()
```

Muestra el estado actual del vehículo.

Returns:

una cadena con los kilómetros recorridos y el combustible que hay en el depósito

viajar

```
public String viajar(double kmViajar)
```

Realiza un viaje de los kilómetros indicados. Suma los kilómetros al total y gasta combustible según el consumo del vehículo (0,07 l/km). Si no hay combustible suficiente, el viaje no se realiza.

Parameters:

kmViajar - kilómetros que se quieren recorrer (debe ser mayor que 0)

Returns:

un mensaje con el resultado del viaje: kilómetros realizados, combustible restante y kilómetros totales; o un mensaje de error si los kilómetros no son válidos o no hay combustible suficiente

repostar

```
public String repostar(double litros)
```

Reposta una cantidad de litros y suma su coste al dinero gastado. No se puede repostar más de lo que cabe en el depósito.

Parameters:

litros - litros a repostar (debe ser mayor que 0)

Returns:

un mensaje con los litros repostados, el coste de esta operación y el total gastado; o un mensaje de error si los litros no son válidos o no caben en el depósito

llenar

```
public String llenar()
```

Llena el depósito hasta su capacidad máxima y suma el coste de los litros añadidos al dinero gastado.

Returns:

un mensaje con el combustible que hay tras llenar, el coste de esta operación y el total gastado

actualizarPrecio

```
public static String actualizarPrecio(double nuevoPrecio)
```

Actualiza el precio del litro de combustible. Al ser un método estático, el cambio afecta a todos los vehículos.

Parameters:

nuevoPrecio - el nuevo precio por litro (debe ser mayor que 0)

Returns:

un mensaje con el nuevo precio, o un mensaje de error si el precio no es válido

mostrarPropietario

```
public String mostrarPropietario()
```

Muestra el propietario del vehículo.

Returns:

una cadena con el nombre y el NIF del propietario